

CAHIER DES CHARGES — DOCUMENT TECHNIQUE

Serveur SQL MCP

« SQL Explorer » — Exploration sécurisée d'une base de données
par un agent d'intelligence artificielle

Projet : sql-mcp-server | Version du document : 1.0

Document rédigé le 4 août 2026

Destiné à : présentation générale du projet, exigences et fonctionnement

Sommaire

1. Introduction — qu'est-ce que ce projet ?
2. Objectifs du projet
3. Fonctionnalités — les six outils exposés
4. Logique de fonctionnement
5. Exigences de sécurité — la défense en profondeur
6. Modèle de données
7. Technologies utilisées
8. Exigences non fonctionnelles
9. Structure du projet
10. Tests et validation
11. Installation — ce qui a été réalisé sur ce poste
12. Comment tester le serveur — résultats attendus
13. Glossaire

1 Introduction — qu'est-ce que ce projet ?

Ce projet s'appelle **sql-mcp-server**. En une phrase : c'est un programme qui permet à un assistant d'intelligence artificielle (comme Claude) de **lire et d'interroger une base de données** tout seul, en langage naturel, sans jamais pouvoir la modifier ou l'endommager.

1.1 Le protocole MCP, en langage simple

MCP signifie *Model Context Protocol* (« protocole de contexte pour les modèles »). C'est une norme créée pour standardiser la façon dont une IA se connecte à des outils extérieurs — une base de données, un système de fichiers, une API, etc.

Analogie : le protocole MCP joue le même rôle qu'une prise USB. Peu importe l'appareil branché (clavier, disque dur, imprimante), la prise est toujours la même. De la même façon, peu importe l'outil externe branché à l'IA, la façon dont l'IA « parle » avec lui est toujours la même grâce au MCP.

1.2 Ce que fait précisément ce serveur

Ce serveur MCP se branche sur une base de données **PostgreSQL** contenant des données fictives de vente en ligne (clients, produits, commandes). Il expose **6 outils** que l'IA peut utiliser pour explorer cette base et répondre à des questions comme :

- « *Quels sont les clients ayant passé plus de 5 commandes le mois dernier ?* »
- « *Quelle catégorie de produits génère le plus de revenus ?* »

L'IA explore la structure de la base, compose elle-même les requêtes nécessaires, et obtient les résultats — sans qu'un humain n'écrive la moindre ligne de code SQL.

Point essentiel : ce serveur est **strictement en lecture seule**. L'IA peut consulter les données, jamais les modifier, les ajouter ou les supprimer. Cette garantie est au cœur de tout le projet (voir section 5).

2 Objectifs du projet

- **Autonomie de l'IA :** permettre à un agent d'explorer une base de données inconnue par lui-même (voir sa structure, ses tables, ses colonnes) avant d'écrire une requête.
- **Sécurité absolue :** garantir qu'aucune écriture (ajout, modification, suppression) ne puisse survenir, que ce soit par erreur de l'IA ou par une tentative malveillante glissée dans une requête.
- **Fiabilité :** empêcher qu'une requête mal formée ou trop lourde ne bloque ou ne ralentisse le serveur de base de données.
- **Pédagogie / démonstration :** fournir un jeu de données réaliste (environ 600 clients, 3 500 commandes) pour tester des questions d'analyse de données représentatives d'un cas réel.

3 Fonctionnalités — les six outils exposés

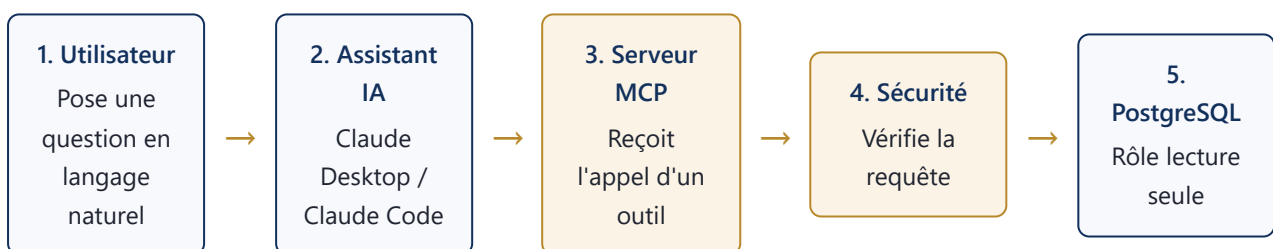
Un « outil » MCP est une fonction que l'IA peut décider d'appeler pendant la conversation, un peu comme elle utiliserait une calculatrice. Voici les six outils que ce serveur met à sa disposition :

Outil	Ce qu'il fait	Exemple d'utilisation par l'IA
<code>list_tables()</code>	Donne la liste de toutes les tables de la base, avec leur description, leur taille et leur nombre de colonnes.	Première étape : « quelles données sont disponibles ? »

<code>describe_table(table)</code>	Détaille les colonnes, leurs types, et les liens (clés étrangères) d'une table précise.	« De quoi est composée la table <code>orders</code> ? »
<code>search_schema(mot-clé)</code>	Cherche dans tous les noms de tables et de colonnes un mot-clé donné.	« Où est stocké l'email des clients ? »
<code>sample_rows(table, limite)</code>	Affiche quelques lignes réelles d'une table (5 par défaut) pour voir à quoi ressemblent les données.	« Montre-moi 3 exemples de clients. »
<code>count_rows(table)</code>	Retourne le nombre total de lignes d'une table.	« Combien de commandes au total ? »
<code>execute_select(requête SQL)</code>	Exécute une requête <code>SELECT</code> libre écrite par l'IA et renvoie les résultats.	« Calcule le revenu total par catégorie. »

4 Logique de fonctionnement

Voici le trajet complet d'une question posée par un utilisateur, de la question jusqu'à la réponse :



1. L'utilisateur pose une question à l'assistant IA (ex. : « quel pays génère le plus de revenu par commande ? »).
2. L'IA comprend qu'elle a besoin de données. Elle décide d'appeler `list_tables()` puis `describe_table()` pour comprendre la structure de la base.
3. L'IA rédige elle-même une requête SQL et appelle `execute_select(sql)`.
4. Le serveur reçoit cette requête et la fait passer par une série de **vérifications de sécurité** avant de l'exécuter (voir section 5).
5. Si la requête est jugée sûre, elle est envoyée à PostgreSQL, exécutée avec un rôle qui n'a **que le droit de lecture**, avec une limite de temps et un nombre maximal de lignes.
6. Le résultat remonte au serveur, puis à l'IA, qui formule une réponse en français (ou dans la langue de l'utilisateur) à partir des données obtenues.

5 Exigences de sécurité — la défense en profondeur

Le point le plus délicat de ce projet est de permettre à une IA d'écrire des requêtes SQL **librement**, sans jamais lui laisser la possibilité de nuire à la base de données. Le projet répond à ce défi avec **cinq couches de protection indépendantes** : si l'une d'elles est contournée ou contient un bug, les autres empêchent quand même le dégât.

- 1 **Un rôle PostgreSQL dédié en lecture seule** (`mcp_readonly`). Ce compte de base de données n'a techniquement **aucun droit d'écriture** — même si toutes les autres protections échouaient, la base de données elle-même refuserait toute modification.
- 2 **Une transaction en lecture seule à chaque connexion** (`SET TRANSACTION READ ONLY`). Une seconde barrière, indépendante de la première, imposée à chaque requête.
- 3 **Validation de la requête SQL elle-même** : le serveur vérifie qu'il s'agit d'une seule instruction `SELECT` (ou `WITH ... SELECT`), qu'elle ne contient pas plusieurs instructions cachées derrière un point-virgule, qu'elle ne contient aucun commentaire SQL (une technique classique pour dissimuler du code malveillant), et qu'aucun mot-clé dangereux n'apparaît (`INSERT` , `UPDATE` , `DELETE` , `DROP` , `GRANT` , etc.).
- 4 **Validation stricte des noms de tables**. Lorsqu'un nom de table est demandé (ex. `describe_table("orders")`), il est vérifié à la fois par un motif de caractères autorisés et par comparaison avec la liste réelle des tables existantes, récupérée en direct depuis la base.
- 5 **Limites de ressources**. Chaque requête a un délai maximal d'exécution (5 secondes par défaut) et un nombre maximal de lignes renvoyées (500 par défaut), même si la requête de l'IA ne précise aucune limite elle-même.

Exemple concret testé : si l'IA (ou un utilisateur malintentionné) tente `SELECT * FROM customers; DROP TABLE customers;` , le serveur renvoie immédiatement une erreur claire (« plusieurs instructions ne sont pas autorisées ») **sans jamais contacter la base de données**. Cela a été vérifié en conditions réelles lors du déploiement de ce projet.

6 Modèle de données

La base de données fournie en exemple simule une boutique en ligne. Elle contient cinq tables liées entre elles :



Table	Contenu
categories	Catégories de produits (Électronique, Maison, Livres, Sport, Beauté, Jouets).
products	Catalogue des produits en vente, chacun lié à une catégorie et avec un prix unitaire.
customers	Clients inscrits (environ 600), avec nom, email, date d'inscription et pays.
orders	Commandes passées par les clients (environ 3 500), avec date et statut (complétée, annulée, remboursée...).
order_items	Lignes de détail de chaque commande (quel produit, quelle quantité, à quel prix). Le revenu d'une commande se calcule ainsi : quantité × prix unitaire.

Les données sont volontairement un peu « imparfaites » (quelques emails manquants, de rares commandes en très grande quantité) afin que les questions d'analyse ressemblent à un cas réel plutôt qu'à un jeu de données artificiellement parfait.

7 Technologies utilisées

Composant	Technologie	Rôle dans le projet
Langage de programmation	Python (3.14)	Langage dans lequel tout le serveur est écrit.
Base de données	PostgreSQL (16)	Système de gestion de base de données relationnelle qui stocke les données de vente.
Protocole de communication	MCP — Model Context Protocol (SDK 2.0)	Standard qui permet à l'assistant IA de découvrir et d'appeler les outils du serveur.
Connecteur base de données	psycopg2	Bibliothèque Python qui permet d'envoyer des requêtes SQL à PostgreSQL.
Génération de données de test	Faker	Bibliothèque qui génère des noms, emails et adresses réalistes mais fictifs.
Tests automatisés	pytest, pytest-asyncio	Vérifient automatiquement que la logique de sécurité et les outils fonctionnent correctement.

Conteneurisation (optionnelle)	Docker / docker-compose	Méthode prévue à l'origine pour démarrer PostgreSQL facilement ; une installation native de PostgreSQL a été utilisée à la place sur ce poste.
Outil de diagnostic	MCP Inspector (Node.js)	Interface web qui permet de tester manuellement les outils du serveur sans passer par une IA.

8 Exigences non fonctionnelles

Exigence	Détail
Performance	Chaque requête est limitée à 5 secondes d'exécution et 500 lignes de résultat par défaut (valeurs modifiables).
Sécurité	Cinq couches de protection indépendantes (voir section 5) ; aucune écriture possible en base.
Portabilité	Fonctionne aussi bien avec une base PostgreSQL démarrée via Docker qu'avec une installation native (comme configuré ici).
Testabilité	39 tests automatisés couvrent la logique de sécurité et le comportement des outils, exécutables sans base de données réelle (via simulation).
Configuration	Tous les paramètres (adresse de la base, identifiants, limites) se règlent via des variables d'environnement, sans toucher au code.

9 Structure du projet

```

sql_mcp_server/
  config.py      # Paramètres lus depuis les variables d'environnement
  security.py    # Toute la logique de validation (le cœur de la sécurité)
  db.py         # Accès à PostgreSQL (requêtes techniques)
  server.py      # Définition des 6 outils exposés à l'IA
scripts/
  init_schema.sql      # Création des tables + rôle lecture seule
  generate_sample_data.py  # Génère les données factices (clients, commandes...)
tests/
  test_security.py    # 18+ scénarios : injection SQL, contournements, etc.
  test_tools.py       # Comportement des 6 outils

```

10 Tests et validation

Le projet inclut une suite de **39 tests automatisés** qui vérifient, sans avoir besoin d'une vraie base de données connectée, que :

- toute tentative d'injection SQL est bien rejetée (point-virgule caché, commentaires, mots-clés interdits) ;
- les noms de table invalides ou inconnus sont refusés ;
- chaque outil se comporte comme attendu face à des cas normaux et à des cas limites.

Vérifications effectuées lors du déploiement sur ce poste : au-delà des tests automatisés, le serveur a été connecté à une vraie base de données PostgreSQL et interrogé en conditions réelles : les 6 outils ont été appelés avec succès (liste des tables, description d'une table, échantillon de lignes, requête d'agrégation par pays/catégorie), et une tentative d'attaque (`DELETE` , requêtes empilées) a bien été bloquée sans toucher aux données.

11 Installation — ce qui a été réalisé sur ce poste

Le projet prévoyait à l'origine d'utiliser **Docker** pour démarrer PostgreSQL automatiquement. Docker n'étant pas installé sur cette machine, une **installation PostgreSQL déjà présente** (utilisée par ailleurs avec pgAdmin) a été réutilisée à la place. Étapes réalisées :

1. Création d'une base de données nommée `sales` .
2. Exécution du script `init_schema.sql` : création des 5 tables et du rôle de lecture seule `mcp_readonly` .
3. Génération des données factices (600 clients, 3 500 commandes) via le script Python fourni.
4. Installation des dépendances Python du projet dans un environnement virtuel dédié (`venv`).
5. Copie du fichier de configuration `.env.example` vers `.env` .
6. Exécution de la suite de tests (39/39 réussis).
7. Connexion réelle au serveur via **MCP Inspector**, un outil de diagnostic qui simule un client IA pour tester chaque outil manuellement dans un navigateur.

12 Comment tester le serveur — résultats attendus

Le serveur communique en `stdio` (entrées/sorties standard) : il n'a pas d'interface visuelle par lui-même et n'est pas destiné à être lancé « à la main » dans un terminal pour y taper des

commandes. Il est conçu pour être piloté par un **client MCP**. Deux façons de le tester ont été utilisées :

12.1 Méthode A — MCP Inspector (interface visuelle)

MCP Inspector est un outil qui simule un client IA dans une page web, pour tester chaque outil à la main, sans écrire de code.

1. Lancer la commande `npx @modelcontextprotocol/inspector` suivie du chemin complet vers l'exécutable Python du projet et de l'argument `-m sql_mcp_server.server`, depuis le dossier du projet.
2. Une page web s'ouvre automatiquement (adresse du type `http://localhost:6274/...`), avec un jeton de sécurité déjà inclus dans le lien.
3. Renseigner les variables d'environnement de connexion à la base (`PGHOST` , `PGPORT` , `PGDATABASE` , `PGUSER` , `PGPASSWORD`) puis cliquer sur **Connect**.
4. Aller dans l'onglet **Tools**, choisir un outil dans la liste, remplir ses paramètres si besoin, puis cliquer sur **Run**.

Signe que la connexion fonctionne : le message d'erreur rouge (« Connection Error ») disparaît, le bouton passe de *Connect* à *Disconnect*, et l'onglet **Tools** affiche les 6 outils disponibles. Si une erreur de chemin apparaît (« le chemin d'accès spécifié est introuvable »), c'est que le champ *Command* contient un chemin relatif au lieu d'un chemin complet — il faut alors indiquer le chemin absolu vers `python.exe` .

12.2 Méthode B — appel direct des outils (vérification technique)

Pour une vérification plus rigoureuse, un petit programme Python peut se connecter au serveur exactement comme le ferait un vrai client MCP (même protocole, mêmes échanges de messages), appeler les outils, et afficher les résultats bruts. C'est cette méthode qui a servi à valider le projet ligne par ligne.

12.3 Résultats attendus, outil par outil

Voici, pour chaque test, la sortie réellement obtenue lors de la validation de ce projet — à utiliser comme référence pour savoir si « ça marche ».

Test 1 — `list_tables()`

Doit renvoyer les 5 tables de la base, avec leur taille et leur nombre de colonnes :

```
[
  { "table_name": "categories", "description": "Product categories...", "size": "48 kB", "c
  { "table_name": "customers", "description": "Registered customers.", "size": "176 kB", "c
  { "table_name": "order_items", "description": "Line items...", "size": "768 kB", "co
  { "table_name": "orders", "description": "One row per order.", "size": "344 kB", "co
```

```
{ "table_name": "products", "description": "Catalog of products.", "size": "32 kB", "co
}
```

Test 2 — `count_rows("orders")`

Doit renvoyer le nombre total de commandes générées :

```
{ "table": "orders", "row_count": 3500 }
```

Test 3 — `execute_select(...)` avec une vraie question d'analyse

Exemple : revenu moyen par commande, par pays. Doit renvoyer 8 lignes (une par pays) triées par valeur décroissante, sans erreur :

```
{
  "row_count": 8,
  "rows": [
    { "country": "Germany", "orders": 442, "avg_order_value": "1164.43" },
    { "country": "Morocco", "orders": 399, "avg_order_value": "1147.67" },
    ... (6 autres pays)
  ]
}
```

Test 4 — tentative d'attaque (doit être refusée)

Exemple : `SELECT * FROM customers; DROP TABLE customers;`

Un fonctionnement correct renvoie une **erreur explicite**, sans toucher à la base :

```
{ "error": "Multiple statements are not allowed" }
```

Comment savoir que le test global est réussi : les tests 1 à 3 doivent renvoyer des **données** (pas de champ `"error"`), et le test 4 doit renvoyer un champ `"error"` explicite **sans qu'aucune ligne de la base ne disparaisse** (vérifiable en relançant `count_rows("orders")` après coup : le nombre doit rester identique, ici 3500).

12.4 Tests automatisés (rappel)

En complément des vérifications manuelles ci-dessus, la commande `pytest` exécutée à la racine du projet doit afficher, si tout fonctionne :

```
===== 39 passed in 2.83s =====
```

Un nombre différent de tests réussis (ou un test en échec, affiché en rouge) indique un problème dans le code de validation ou dans la configuration de l'environnement, à corriger avant de considérer le serveur comme fiable.

13 Glossaire

MCP (Model Context Protocol)

Norme standardisée qui permet à une IA de se connecter à des outils ou données externes.

LLM (Large Language Model)

Modèle de langage de grande taille — le type de technologie derrière les assistants IA comme Claude, capable de comprendre et générer du texte, et de décider quand appeler un outil.

SQL

Langage utilisé pour interroger une base de données relationnelle (ex. `SELECT * FROM clients`).

SELECT / lecture seule

Type de requête SQL qui ne fait que *lire* des données, sans jamais les modifier.

Injection SQL

Technique d'attaque consistant à glisser du code SQL malveillant dans une requête pour contourner les protections et endommager une base de données.

Rôle (base de données)

Compte utilisateur PostgreSQL avec des droits précis (ici : uniquement le droit de lecture).

Environnement virtuel (venv)

Dossier isolé qui contient une installation Python propre au projet, pour éviter les conflits avec d'autres logiciels de l'ordinateur.

Docker

Outil qui permet de faire tourner un logiciel (ici PostgreSQL) dans un environnement isolé et préconfiguré, sans avoir à l'installer directement sur l'ordinateur.

Timeout

Délai maximal autorisé avant qu'une opération ne soit automatiquement annulée.